## ADR-1. Vue

**## Situación**
Aceptado

**## Contexto**
La plataforma requiere una interfaz de usuario interactiva, modular y fácil de mantener para el MVP. Es necesario soportar flujos como el onboarding técnico, el feed de proyectos, la visualización del perfil del usuario y la gestión de postulaciones, sin introducir complejidad innecesaria en la capa de presentación. Además, la arquitectura exige que la experiencia de usuario sea reactiva y responda de forma inmediata a cambios en los datos del perfil y del estado del proyecto.

**## Decisión**
Se selecciona Vue.js como framework de componentes para la capa de presentación. Su modelo basado en reactividad declarativa permite construir interfaces interactivas de forma clara y mantener un flujo de actualización consistente entre el estado del usuario y la vista. Esta decisión favorece la táctica de usabilidad Maintain System Model, al permitir que la interfaz refleje de forma inmediata cambios en el perfil, las recomendaciones y los estados del proyecto. Se descarta React por la restricción arquitectónica del proyecto, y se descarta Angular por su mayor complejidad inicial y curva de aprendizaje para un MVP.

**## Consecuencias**
- **Ventajas:** Facilita la construcción de interfaces reactivas y coherentes con los flujos principales del sistema.
- **Ventajas:** Reduce el tiempo de implementación del MVP al ofrecer un modelo de desarrollo más directo para pantallas transaccionales.
- **Desventajas:** El ecosistema de librerías UI y componentes listos para usar es menor que en otros frameworks.
- **Desventajas:** El equipo debe alinearse con la sintaxis y el modelo de reactividad propio de Vue.

**## Cumplimiento**
- **Auditoría de código:** Verificar que las vistas principales utilicen componentes reactivos de Vue y que no se manipule directamente el DOM de forma imperativa.
- **Pruebas funcionales**: Validar que los cambios en el perfil actualicen automáticamente el feed de recomendaciones y que los flujos de publicación y postulación reflejen correctamente los cambios de estado en la interfaz.

## Notas

**Autor:** Sebastián Ávila (s.avila03@ufromail.cl)
**Autor:** Carlos Moris (c.moris02@ufromail.cl)
**Autor:** Fernando Robles (f.robles03@ufromail.cl)
**Día de publicación:** 17-05-2026
**Última actualización:** 06-07-2026

## ADR-2. Tailwind CSS

## Situación
Aceptado

## Contexto
El desarrollo del MVP exige construir múltiples interfaces altamente interactivas y visualmente coherentes, como el formulario de onboarding, el perfil del usuario, el feed personalizado y las vistas de proyectos. Para asegurar una experiencia de usuario consistente y una implementación ágil, es necesario contar con un sistema de estilos que permita construir interfaces rápidamente sin generar deuda visual ni invertir tiempo excesivo en diseño desde cero.

## Decisión
Se selecciona Tailwind CSS como framework de estilos utilitario para la capa de presentación. Su enfoque basado en clases predefinidas acelera la implementación de pantallas complejas y garantiza consistencia visual en espaciados, colores, tipografías y estados interactivos. Esto resulta especialmente útil para cumplir con los objetivos de usabilidad del sistema en un MVP. Se descarta Bootstrap por su enfoque visual más genérico, y CSS puro por su costo de mantenimiento y desarrollo más alto.

## Consecuencias
- **Ventajas:** Acelera el desarrollo de interfaces y mejora la consistencia visual entre pantallas.
- **Ventajas:** Facilita la implementación iterativa del MVP sin depender de un sistema de diseño complejo.
- **Desventajas**: Puede generar una gran cantidad de clases en el marcado, reduciendo la legibilidad en algunos componentes.
- **Desventajas:** El equipo debe dominar el catálogo de clases utilitarias para evitar sobrecarga y mantener la calidad del código.

## Cumplimiento
- **Auditoría de código:** Verificar que la estilización se realice exclusivamente mediante clases utilitarias de Tailwind y que no se utilicen estilos inline o CSS personalizado de forma redundante.
- **Pruebas de interfaz:** Evaluar visualmente que los componentes de onboarding, perfil y feed mantengan una apariencia uniforme y coherente.

## Notas

**Autor:** Sebastián Ávila (s.avila03@ufromail.cl)
**Autor:** Carlos Moris (c.moris02@ufromail.cl)
**Autor:** Fernando Robles (f.robles03@ufromail.cl)
**Día de publicación:** 17-05-2026
**Última actualización:** 06-07-2026

## ADR-3. Node.js

**## Situación**
Aceptado

**## Contexto**
El backend de la plataforma debe procesar operaciones concurrentes relacionadas con autenticación, consultas a base de datos, procesamiento de colas de trabajo, envío de correos y manejo de archivos. Además, el sistema debe mantenerse resiliente frente a fallos o demoras en servicios externos sin bloquear el flujo principal de la aplicación. Para ello, se requiere un runtime que soporte de forma eficiente un modelo asíncrono de E/S.

**## Decisión**
Se selecciona Node.js con TypeScript como entorno de ejecución principal del backend. Su modelo basado en event loop no bloqueante permite manejar múltiples operaciones de entrada/salida de forma eficiente, lo que resulta adecuado para la arquitectura del sistema. Esta elección facilita la implementación de procesos asíncronos, la coordinación de servicios externos y la integración con colas de trabajo. Se descarta Python con FastAPI por la fragmentación del stack y la mayor complejidad operativa que implicaría para el MVP.

**## Consecuencias**
- **Ventajas:** Soporta de forma natural el procesamiento asíncrono de operaciones de negocio e integración.
- **Ventajas:** Permite unificar el stack tecnológico entre frontend y backend, reduciendo la fricción en el desarrollo.
- **Desventajas:** La implementación de lógica computacional intensiva debe evaluarse con cuidado, ya que Node.js no es la opción más natural para ciertos escenarios de procesamiento masivo.
- **Desventajas:** Requiere disciplina en el manejo de promesas, control de errores y diseño asíncrono.

**## Cumplimiento**
- **Auditoría de código:** Verificar que las operaciones de base de datos, correo, storage y colas se implementen mediante Promises y async/await, evitando bloqueos innecesarios.
- **Pruebas de tolerancia a fallos:** Validar que la indisponibilidad temporal de servicios externos no comprometa la disponibilidad del backend ni la experiencia de usuario principal.

## Notas

**Autor:** Sebastián Ávila (s.avila03@ufromail.cl)
**Autor:** Carlos Moris (c.moris02@ufromail.cl)
**Autor:** Fernando Robles (f.robles03@ufromail.cl)
**Día de publicación:** 17-05-2026
**Última actualización:** 06-07-2026

## ADR-4. NestJS

**## Situación**
Aceptado

**## Contexto**
La API REST del backend debe soportar de forma modular las tácticas de seguridad definidas, así como la separación por módulos funcionales como usuarios, proyectos, postulaciones, financiamiento y afinidad. Express.js fue descartado porque su naturaleza minimalista obliga a construir manualmente toda la estructura arquitectónica, aumentando el riesgo de omitir decisiones de seguridad y organización en rutas críticas.

**## Decisión**
Se adopta NestJS como framework HTTP del backend. Su arquitectura de guards, interceptores e inyección de dependencias permite mapear directamente las tácticas de seguridad y organización funcional del sistema. La separación en módulos funcionales garantiza bajo acoplamiento entre componentes y facilita la extensión del sistema.

**## Consecuencias**
- **Ventajas:** Los guards e interceptores evitan omisiones accidentales de seguridad; la inyección de dependencias facilita el testing por componente.
- **Ventajas:** La estructura modular es predecible y reduce fricción para nuevos integrantes.
- **Desventajas:** Mayor curva de aprendizaje que Express y boilerplate inicial significativo.

**## Cumplimiento**
- Todos los endpoints de recursos privados deben tener guardias de autenticación y autorización aplicados.
- La validación de entradas debe configurarse de forma global.
- Las operaciones sensibles deben registrar eventos de auditoría cuando corresponda.

## Notas

Autor: Sebastián Ávila (s.avila03@ufromail.cl)
Autor: Carlos Moris (c.moris02@ufromail.cl)
Autor: Fernando Robles (f.robles03@ufromail.cl)
Día de publicación: 17-05-2026
Última actualización: 06-07-2026

## ADR-5. Passport.js

**## Situación**
Aceptado

**## Contexto**
El sistema debe permitir la autenticación segura de usuarios y empresas mediante credenciales propias o proveedores OAuth externos como Google. Siendo la seguridad un atributo prioritario, es necesario un mecanismo flexible que no introduzca dependencias críticas externas en la ruta de inicio de sesión.

**## Decisión**
Se selecciona Passport.js como la biblioteca de autenticación para el backend. Su arquitectura basada en estrategias intercambiables soporta directamente la táctica Authenticate Actors, permitiendo agregar nuevos proveedores de autenticación en el futuro sin modificar la lógica de negocio central. Se combina con JWT para la gestión de sesiones entre cliente y servidor.

**## Consecuencias**
- **Ventajas:** Facilita la autenticación local y externa de forma modular.
- **Ventajas:** Permite separar claramente la verificación de identidad de la autorización.
- **Desventajas:** Incrementa la complejidad inicial de implementación en comparación con soluciones más cerradas.

**## Cumplimiento**
- Verificar que el pipeline de autenticación utilice estrategias de Passport de forma aislada.
- Comprobar que la validación de los tokens JWT se realice correctamente.

**## Notas**
Autor: Sebastián Ávila (s.avila03@ufromail.cl)
Autor: Carlos Moris (c.moris02@ufromail.cl)
Autor: Fernando Robles (f.robles03@ufromail.cl)
Día de publicación: 17-05-2026
Última actualización: 06-07-2026

## ADR-6. BullMQ

**## Situación**
Aceptado

**## Contexto**
El sistema requiere ejecutar operaciones de forma asíncrona para no bloquear la respuesta principal de la API. Ejemplos de ello son el recálculo de afinidad, el envío de notificaciones, la gestión de estados de proyectos y otras tareas de integración que pueden presentar latencias o fallos temporales.

**## Decisión**
Se selecciona BullMQ como mecanismo de gestión de colas de trabajo, utilizando Redis como backend de persistencia y coordinación. BullMQ permite desacoplar tareas costosas o sensibles al tiempo de la respuesta HTTP principal, manteniendo la arquitectura del sistema más resiliente frente a fallas temporales. La estrategia de reintentos y la ejecución en workers independientes favorecen la interoperabilidad y la tolerancia a fallos. Se descarta RabbitMQ por su complejidad de configuración para las necesidades del MVP.

**## Consecuencias**
- **Ventajas:** Mejora la capacidad del sistema para procesar tareas en segundo plano sin bloquear la API principal.
- **Ventajas:** Facilita la implementación de reintentos, manejo de fallos y procesamiento resiliente.
- **Desventajas:** Introduce un componente adicional de infraestructura (Redis) que debe mantenerse y monitorearse.
- **Desventajas:** Requiere diseño cuidadoso para evitar tareas demasiado grandes o dependencias mal encapsuladas.

**## Cumplimiento**
- **Pruebas de integración:** Validar que las tareas encoladas se procesen correctamente en segundo plano.
- **Pruebas de resiliencia:** Simular fallos temporales y verificar que BullMQ mantenga los trabajos y los reintente de forma controlada.

## Notas

Autor: Sebastián Ávila (s.avila03@ufromail.cl)
Autor: Carlos Moris (c.moris02@ufromail.cl)
Autor: Fernando Robles (f.robles03@ufromail.cl)
Día de publicación: 17-05-2026
Última actualización: 06-07-2026

## ADR-7. PostgreSQL

**## Situación**
Aceptado

**## Contexto**
Las entidades principales del sistema como usuarios, proyectos y postulaciones tienen relaciones explícitas entre sí con semántica precisa. El cálculo del índice de afinidad requiere comparar roles e intereses del perfil del usuario contra los requerimientos de cada proyecto disponible simultáneamente, lo que implica joins y filtros compuestos sobre múltiples tablas. Adicionalmente, las operaciones de aceptación de postulaciones deben ejecutarse como procesos transaccionales y consistentes.

**## Decisión**
Se adopta PostgreSQL como sistema de gestión de base de datos relacional principal. Su soporte para transacciones ACID, joins complejos y consultas relacionales lo hace adecuado para la lógica de negocio del sistema. Se descarta MongoDB por no ajustarse tan bien al modelo relacional y a las relaciones explícitas del dominio.

**## Consecuencias**
- **Ventajas:** Facilita la integridad transaccional y la consulta relacional de datos de negocio.
- **Ventajas:** Mejora el rendimiento y la claridad de las consultas sobre estructuras con relaciones complejas.
- **Desventajas:** Requiere migraciones explícitas para cambios estructurales del modelo de datos.
- **Desventajas:** Es menos flexible que un modelo documental para cambios de esquema muy frecuentes.

**## Cumplimiento**
- Verificar que las operaciones críticas de postulación y estados de proyecto se ejecuten dentro de transacciones consistentes.
- Validar que las consultas de afinidad y filtrado de feed se ejecuten de forma eficiente sobre la base relacional.

## Notas

Autor: Sebastián Ávila (s.avila03@ufromail.cl)
Autor: Carlos Moris (c.moris02@ufromail.cl)
Autor: Fernando Robles (f.robles03@ufromail.cl)
Día de publicación: 17-05-2026
Última actualización: 06-07-2026

## ADR-8. MinIO / S3 Compatible Object Storage

**## Situación**
Aceptado

**## Contexto**
La plataforma debe gestionar archivos como CVs, imágenes de proyectos y documentos adjuntos. Estos recursos requieren almacenamiento confiable, acceso controlado y una forma sencilla de generar URLs públicas o privadas según el caso.

**## Decisión**
Se adopta MinIO como solución de almacenamiento de objetos compatible con S3. Esta decisión permite manejar archivos de forma desacoplada del servicio principal y facilita la carga, descarga y eliminación de recursos multimedia. Se descarta almacenar los archivos directamente en el sistema de archivos del contenedor o en la base de datos, dado que ambas alternativas complican la gestión y escalabilidad de recursos multimedia.

**## Consecuencias**
- **Ventajas:** Permite almacenar archivos de forma organizada y escalable.
- **Ventajas:** Desacopla los recursos multimedia del núcleo del sistema.
- **Desventajas:** Requiere infraestructura adicional de almacenamiento.
- **Desventajas:** La política de acceso debe diseñarse cuidadosamente.

**## Cumplimiento**
- Verificar que la subida, descarga y eliminación de archivos funcione correctamente.
- Confirmar que los recursos se accedan mediante URLs correctas y con permisos adecuados.

**## Notas**
Autor: Sebastián Ávila (s.avila03@ufromail.cl)
Autor: Carlos Moris (c.moris02@ufromail.cl)
Autor: Fernando Robles (f.robles03@ufromail.cl)
Día de publicación: 17-05-2026
Última actualización: 06-07-2026

## ADR-9. JSON Web Tokens (JWT)

**## Situación**
Aceptado

**## Contexto**
El sistema requiere mantener la identidad del usuario autenticado entre requests sucesivos a la API REST del backend. Dado que la arquitectura aplica un modelo desacoplado entre frontend y backend, es necesario un mecanismo que permita verificar la identidad del usuario in cada request sin introducir una sesión compartida compleja.

**## Decisión**
Se adopta JSON Web Tokens (JWT) como mecanismo de gestión de sesiones, utilizado en conjunto con Passport.js. El servidor firma un token con la identidad del usuario y lo entrega al cliente, que lo envía en el header de autorización en cada request subsiguiente. Esta solución es coherente con la arquitectura actual y facilita la validación de identidad en endpoints protegidos.

**## Consecuencias**
- **Ventajas:** Reduce la necesidad de mantener estado de sesión en el servidor.
- **Ventajas:** Facilita la autenticación en una arquitectura desacoplada.
- **Desventajas:** La revocación de tokens requiere políticas adicionales.
- **Desventajas:** La seguridad del sistema depende de la correcta gestión de claves y expiración.

**## Cumplimiento**
- Verificar que los endpoints protegidos validen correctamente los JWT.
- Confirmar que los tokens con firma inválida o expirados sean rechazados.

**## Notas**
Autor: Sebastián Ávila (s.avila03@ufromail.cl)
Autor: Carlos Moris (c.moris02@ufromail.cl)
Autor: Fernando Robles (f.robles03@ufromail.cl)
Día de publicación: 17-05-2026
Última actualización: 06-07-2026

## ADR-10. Nodemailer / SMTP para notificaciones por correo

**## Situación**
Aceptado

**## Contexto**
La plataforma requiere enviar notificaciones por correo sobre eventos del flujo de negocio, como registro, postulaciones, aceptación o rechazo de postulaciones y cambios de estado. Estas notificaciones deben ser entregadas de forma confiable y sin bloquear el procesamiento principal del sistema.

**## Decisión**
Se adopta Nodemailer como librería de envío de correos, utilizando un servidor SMTP configurado para la plataforma. Esta decisión permite centralizar la lógica de notificación transaccional en un componente claro y desacoplado, facilitando el envío de correos desde los distintos módulos del backend. Se descarta la implementación manual de envío por socket o la gestión directa de servidores de correo, dado que aumenta la complejidad operativa y reduce la mantenibilidad del sistema.

**## Consecuencias**
- **Ventajas:** Permite enviar notificaciones de forma confiable y consistente desde distintos módulos del sistema.
- **Ventajas:** Mantiene el backend desacoplado de la lógica de transporte de correos y facilita futuras extensiones.
- **Desventajas:** Depende de la disponibilidad y configuración del servicio SMTP.
- **Desventajas:** Requiere manejo adecuado de errores y reintentos para evitar fallos silenciosos en la entrega.

**## Cumplimiento**
- **Pruebas de integración:** Verificar que los correos se envíen correctamente para los eventos de negocio definidos.
- **Auditoría de configuración:** Confirmar que la configuración SMTP sea segura, que se utilicen credenciales adecuadas y que los mensajes sean formateados correctamente.
- **Pruebas de resiliencia:** Simular fallos de envío y confirmar que el sistema registre el evento sin romper el flujo principal.

## ADR-11. DuckDNS para Dominio web

### ## Situación

Aceptado

### ## Contexto

El despliegue del MVP requiere exponer los servicios del frontend y backend en internet mediante dominios accesibles, distribuyendo componentes en una instancia de AWS EC2 y un VPS. Para evitar el costo y la gestión de comprar un dominio personalizado en esta etapa, se necesita un servicio que proporcione subdominios estables y gratuitos, que permitan asignar nombres legibles a las IPs públicas de los servidores y facilitar así la configuración de certificados SSL.

### ## Decisión

Se selecciona DuckDNS como proveedor de dominios y servicio de DNS dinámico (DDNS). Se asignan los subdominios venturia-ec2.duckdns.org y venturia-vps.duckdns.org para apuntar a las respectivas IPs públicas de los entornos de despliegue requeridos.Esta solución facilita el acceso remoto y la configuración de certificados HTTPS sin depender de IPs efímeras o dominios de pago.
Se descarta el uso de nip.io, ya que DuckDNS ofrece subdominios más legibles que no exponen la IP directamente en la URL y permite un manejo más sencillo mediante tokens si la IP del servidor llegase a cambiar.

### ## Consecuencias

- **Ventajas:** Provee dominios accesibles y legibles sin costo para el MVP.
- **Ventajas:** Facilita la emisión de certificados de seguridad (SSL/TLS) mediante Let's Encrypt para HTTPS.
- **Desventajas:** La disponibilidad del dominio depende de un servicio externo gratuito (DuckDNS).

# Cumplimiento

- **Auditoría de configuración:** Verificar que los registros de DuckDNS apunten a las IPs públicas correctas de la instancia EC2 y del VPS.
- **Gestión de secretos:** Asegurar que el token de actualización de DuckDNS se almacene de forma segura en los scripts de actualización de DNS.
- **Pruebas de acceso:** Validar que los dominios resuelvan correctamente y que el tráfico HTTPS funcione mediante los certificados configurados para cada subdominio.

**## Notas**

Autor: Sebastián Ávila (s.avila03@ufromail.cl)
Autor: Carlos Moris (c.moris02@ufromail.cl)
Autor: Fernando Robles (f.robles03@ufromail.cl)
Día de publicación: 17-05-2026
Última actualización: 05-07-2026